Histórico de Prompts Utilizados no Desenvolvimento da Massa de Dados (ERP Petly Club)

Este documento registra a sequência de prompts e diretrizes utilizadas para orientar a inteligência artificial na construção, validação e refinamento do script SQL de povoamento de testes do ERP Petly Club. Nos prompts, o arquivo aparece como `massa_dados_petly_club.sql`; no repositório, ele é `sql/dados/Massa_de_Dados_Completa_-_ERP_Petly_Club.sql` (ver seção 5).

---

1. Prompt Inicial (Criação e Estruturação Global)

Objetivo: Solicitar a criação do script SQL de carga inicial de testes alinhado aos cenários operacionais e requisitos funcionais do ERP.

Prompt: *"Crie uma massa de dados completa em PostgreSQL 16 para o projeto ERP Petly Club no arquivo `massa_dados_petly_club.sql`. O script deve cobrir os cenários descritos no README (seção 17.1) e todos os requisitos funcionais (RF01 a RF37) e não funcionais (RNF01 a RNF10).*

Requisitos para o script:

1. *Limpeza preventiva de tabelas com `TRUNCATE TABLE ... RESTART IDENTITY CASCADE`.*
2. *População de tabelas base: categorias, fornecedores, funcionários (com hashes de senha), clientes e produtos.*
3. *Carga de estoque inicial com múltiplos lotes (incluindo lotes antigos, novos e vencidos).*
4. *Implementação dos cenários operacionais do README 17.1 (vendas balcão PEPS, reposição de estoque, recebimento parcial de compras, pedidos online com entrega, retirada no balcão, perdas/avarias e pagamento parcial de fornecedores).*
5. *Reajuste final automático de todas as `sequences` (`setval`) do banco de dados.*
6. *Uso de transação explicita (`BEGIN;` ... `COMMIT;`)."*

---

2. Prompt de Revisão e Cobertura de Requisitos Vagos

Objetivo: Ajustar datas de referência, recálculos financeiros e inclusão dos requisitos RF15, RF16 e RF32.

Prompt: *"Revise a massa de dados gerada e faça os seguintes ajustes:*

1. *Corrija o recálculo dos valores da Compra 1 e de suas parcelas para refletir o total de 20 unidades a R$ 100,00 cada.*
2. *Ajuste o status das parcelas financeiras considerando que a data de referência dos testes é 27/10/2026.*
3. *Certifique-se de incluir a movimentação de `liberacao_reserva` simultânea à baixa por `venda` nos pedidos virtuais.*
4. *Garanta que o recebimento parcial de compras não infle o estoque com mercadorias não recebidas.*
5. *Adicione novos cenários operacionais específicos para demonstrar:*
    - **RF32:** Aplicação de desconto no pedido com autorização gerencial.
    - **RF15:** Pagamento de um mesmo pedido utilizando duas formas de pagamento distintas (Dinheiro + PIX).
    - **RF16:** Cancelamento de venda no caixa presencial com devolução de produto ao estoque.*

---

3. Prompt de Alinhamento Estrito com o README e DDL (Refinamento Fato/Regra)

Objetivo: Ajustar fidelidade com as regras estritas do DDL e detalhar os cenários do README 17.1 (PEPS em múltiplos lotes, PIX recusado, devolução parcial com perda e fracionamento de `conta_receber`).

Prompt: *"Revise novamente o SQL atual do ERP Petly Club e corrija os pontos abaixo com base na especificação técnica e no DDL:*

1. *Alinhe os cenários da massa de dados com a seção 17.1 do README (venda simultânea da última unidade de um lote/produto e baixa de múltiplos lotes por PEPS).*
2. *Garanta a rastreabilidade estrita da Compra 1: novos recebimentos de compras não devem ser misturados a lotes antigos existentes.*
3. *No Pedido 3, represente o fluxo de tentativa de pagamento via PIX expirado/não pago (`pagamento_pedido` com status `'recusado'`), além de sua reserva e liberação.*

4. *No Pedido 5 (duas formas de pagamento - RF15), insira dois registros independentes na tabela `conta_receber` (um para cada forma de pagamento), atendendo ao RF25 e à modelagem do DDL.*
5. *Refaça o cenário de cancelamento (Pedido 6) para reproduzir o README exatamente: venda no cartão, cancelamento autorizado pela gerência, devolução de 1 item em boas condições (tipo `'devolucao'`) e perda de 1 item avariado (tipo `'perda'`), além do estorno financeiro.*
6. *Demonstre a autorização gerencial de perdas, avarias e descartes de lotes vencidos (RF12).*
7. *Valide a conformidade do script de carga com o PostgreSQL 16 (tipos de dados, domínios de `CHECK`, FKs e atualização de `sequences`)."*

---

4. Estrutura de Prompt para Solicitação do Histórico

Objetivo: Obter a documentação completa dos comandos e diretrizes utilizados nas interações anteriores.

Prompt: *"me mande agora os prompts que foram utilizados"*

5. Validação da massa de dados pela equipe

Objetivo: registrar como a equipe conferiu o script gerado com auxílio de IA e o que difere entre os prompts acima e o script entregue. A IA auxiliou; a responsabilidade pelo resultado é da equipe.

Como o script foi validado

1. Execução em banco vazio no PostgreSQL 16, logo depois do DDL (`petly_club_ddl_limpo.sql`). A carga terminou em `COMMIT`, sem nenhum erro de PK, FK, CHECK ou gatilho.
2. Os 23 testes de integridade que o banco deve recusar (T01 a T23, em `sql/testes/testes_integridade.sql`) foram executados sobre esta massa, e todos terminaram em erro, como esperado.
3. As 10 consultas de auditoria (AU01 a AU10) retornaram 0 linhas: não há pedido sem itens, venda sem baixa no estoque, pagamento que não bate com as contas a receber nem lote vencido com saldo.
4. Os cenários pedidos foram conferidos nos dados: venda que usa dois lotes com validades diferentes (pedido 1, sai primeiro o lote que vence antes), PIX recusado com a reserva liberada (pedido 3), pagamento com duas formas (pedido 5) e cancelamento no caixa com devolução de um item e perda de outro (pedido 6).

Diferenças entre os prompts e o script entregue

Nos prompts	No script entregue	Por quê
Arquivo `massa_dados_petly_club.sql`	`Massa_de_Dados_Completa_-_ERP_Petly_Club.sql`	O arquivo foi renomeado ao entrar no repositório, na pasta `sql/dados/`.
Saída de estoque por “PEPS” (primeiro que entra, primeiro que sai)	Saída por FEFO (primeiro que vence, primeiro que sai)	A regra do projeto é FEFO (RN20, RF11): para pet shop, o que importa é a validade, não a ordem de entrada. O script segue a regra de negócio, e não o termo usado no prompt. No pedido 1, as duas regras levam ao mesmo lote, porque o lote mais antigo é também o que vence antes.
Tabela `pagamento_pedido` (prompt 3, item 3)	Tabela `pagamento`	O nome correto é o do DDL do grupo; o próprio prompt 3 pede alinhamento com o DDL (item 7).

6. Massa histórica de vendas (30/10/2026)

Objetivo: dar ao banco alguns meses de vendas para as consultas gerenciais, os indicadores e o dashboard, sem alterar a massa de cenários das seções 1 a 3.

Pedido feito à IA (Claude): *"Massa de dados com alguns meses de vendas."*

Diretrizes definidas na conversa e conferidas pela equipe

1. Arquivo separado (`Massa_Historico_Vendas_ERP_Petly_Club.sql`), executado depois da massa de cenários, com faixas de id próprias e sem alterar nenhum registro existente nem os ids usados nos testes.
2. Vendas de 01/08 a 19/10/2026, antes dos cenários, nos dois canais, com entregas, retiradas, PIX não pago, descontos autorizados e crédito em até 3x.
3. Estoque simulado dia a dia: toda saída segue FEFO (RN20); os lotes históricos vencem antes dos lotes dos cenários e terminam com saldo zero, para não mudar o resultado deles.
4. O antipulgas só vende em outubro, do lote 5, porque o lote 4 venceu em 30/09 sem vender (cenário 3).
5. Financeiro coerente com a data de referência 27/10/2026: contas a receber somam o valor de cada pagamento autorizado; as compras têm duas parcelas que somam o total.
6. O script que gera os dados (`gerar_massa_historico.py`) fica no repositório, com semente fixa, para que o resultado possa ser reproduzido.

Validação

1. Carga em banco vazio (DDL → massa de cenários → massa histórica) terminou em `COMMIT`, sem erro.
2. Os 23 testes de integridade continuaram terminando em erro, como esperado, e os testes de UPDATE e DELETE continuaram aceitos.
3. As 10 consultas de auditoria continuaram retornando 0 linhas, e os testes internos de cada uma continuaram encontrando o erro proposital.
4. A saída completa da execução está em `evidencias/saida_execucao_completa.txt`.